

Entregable 2. Dockerfiles y archivo Docker Compose

Integrantes: Derly Dayana Garcia Carrillo · Orlando Rojas Narvaez | Docente: Juan Carlos Polania Cortes | Periodo 2026-2
| Repositorio: github.com/OrlandoRojasNa/taller02-rabbitmq-docker

Resumen

Archivo	Qué hace
<code>docker-compose.yml</code>	Único archivo de composición: levanta <code>rabbitmq</code> , <code>productor</code> y <code>consumidor</code> con <code>docker compose up -d --build</code> . Solo publica los puertos 15672 (consola) y 8080 (API). Los servicios se conectan por nombre (<code>rabbitmq</code>), esperan a que el broker esté <code>healthy</code> y el volumen <code>rabbitmq_data</code> conserva los mensajes.
<code>rabbitmq/Dockerfile</code>	Parte de <code>rabbitmq:4.1-management</code> y copia la configuración y las definiciones.
<code>rabbitmq/rabbitmq.conf</code> y <code>definitions.json</code>	Crean el usuario <code>lab</code> y la cola durable <code>cola.sensores</code> al arrancar, sin pasos manuales.
<code>productor/Dockerfile</code> , <code>consumidor/Dockerfile</code>	Multi-etapa: <code>maven:3.9-eclipse-temurin-21</code> compila y <code>eclipse-temurin:21-jre</code> ejecuta. En el equipo no se instala Java ni Maven.

Archivo de composición

```
docker-compose.yml

name: taller02-rabbitmq

services:
  rabbitmq:
    build: ./rabbitmq
    container_name: rabbitmq
    # El nombre del nodo depende del hostname; fijarlo mantiene los datos del volumen entre reinicios
    hostname: rabbitmq
    ports:
      - "15672:15672" # consola de administracion
    volumes:
      - rabbitmq_data:/var/lib/rabbitmq
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "check_port_connectivity"]
      interval: 10s
      timeout: 10s
      retries: 12
      start_period: 20s
      restart: unless-stopped

  productor:
    build: ./productor
    container_name: productor
    ports:
      - "8080:8080" # API REST
```

```
environment:
  SPRING_RABBITMQ_HOST: rabbitmq
  SPRING_RABBITMQ_PORT: 5672
  SPRING_RABBITMQ_USERNAME: lab
  SPRING_RABBITMQ_PASSWORD: lab123
  APP_COLA: cola.sensores
  TZ: America/Bogota
```

```
depends_on:
  rabbitmq:
    condition: service_healthy
restart: unless-stopped
```

```
consumidor:
  build: ./consumidor
  container_name: consumidor
  environment:
    SPRING_RABBITMQ_HOST: rabbitmq
    SPRING_RABBITMQ_PORT: 5672
    SPRING_RABBITMQ_USERNAME: lab
    SPRING_RABBITMQ_PASSWORD: lab123
    APP_COLA: cola.sensores
    TZ: America/Bogota
  depends_on:
    rabbitmq:
      condition: service_healthy
  restart: unless-stopped
```

```
volumes:
  rabbitmq_data:
```

Imagen de RabbitMQ

rabbitmq/Dockerfile

```
FROM rabbitmq:4.1-management

# Configuración propia del taller: definiciones (usuario + cola durable)
COPY rabbitmq.conf /etc/rabbitmq/conf.d/20-taller.conf
COPY definitions.json /etc/rabbitmq/definitions.json
```

rabbitmq/rabbitmq.conf

```
# Carga usuario, permisos y la cola durable al arrancar el nodo.
# Así la cola existe desde el primer arranque, sin crearla a mano en la consola.
load_definitions = /etc/rabbitmq/definitions.json

default_user = lab
default_pass = lab123
```

rabbitmq/definitions.json

```
{
  "users": [
    {
      "name": "lab",
      "password_hash": "fEeM4FooAVANuJhZEHD/h+Py0mA1332QuCbWFRDRtDH6kC8L",
      "hashing_algorithm": "rabbit_password_hashing_sha256",
      "tags": ["administrator"]
    }
  ],
  "vhosts": [
    { "name": "/" }
  ],
  "permissions": [
    { "user": "lab", "vhost": "/", "configure": ".*", "write": ".*", "read": ".*" }
  ],
  "queues": [
    {
      "name": "cola.sensores",
      "vhost": "/",
      "durable": true,
      "auto_delete": false,
      "arguments": {}
    }
  ]
}
```

Imagen del productor

productor/Dockerfile

```
# Etapa 1: compilar con Maven + JDK dentro del contenedor (nada se instala en el equipo)
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
```

```
COPY src ./src
RUN mvn -B -q package

# Etapa 2: imagen liviana solo con el JRE y el .jar
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/productor.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Imagen del consumidor

consumidor/Dockerfile

```
# Etapa 1: compilar con Maven + JDK dentro del contenedor (nada se instala en el equipo)
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q package

# Etapa 2: imagen liviana solo con el JRE y el .jar
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/consumidor.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```